

PRUEBA TÉCNICA ESCRITA — DESARROLLADOR ODOO (v19)

No se evalúa memorización de API. Se evalúa capacidad de leer requerimientos, tomar decisiones de diseño defendibles, razonar sobre seguridad y rendimiento a escala, y explicar el porqué de una decisión — no la construcción de un módulo completo. El candidato puede consultar documentación oficial de Odoo libremente — es una condición normal de trabajo, no una trampa.

Esta es la variante 100% escrita de la prueba. No incluye construcción de módulo ni debugging sobre un entorno corriendo — para esas partes existe una prueba práctica aparte, a usar solo si el proceso de selección lo requiere.

Toda la prueba es específica a Odoo 19.0. Código que solo es válido en 18 o anterior (sintaxis desactualizada, atributos deprecados) se considera error, no estilo alternativo — ver tabla de §1.1.

1. Parte A — Análisis y planificación (sin código)

Enunciado

Un cliente de retail pide un módulo `library.rental` para gestionar el préstamo de material (libros, equipos audiovisuales) a sus empleados, con estas reglas de negocio:

- Cada ítem prestable tiene un código único, una categoría y un estado (disponible, prestado, en_reparacion, perdido).
- Un empleado puede tener varios préstamos activos simultáneos, pero no puede superar un límite configurable por categoría (ej. máx. 3 libros, máx. 1 proyector).
- Todo préstamo tiene fecha de entrega esperada. Si se vence, el sistema debe notificar al responsable de biblioteca y al empleado.
- Solo el rol "Bibliotecario" puede registrar devoluciones o marcar un ítem como perdido. Cualquier empleado puede consultar su propio historial pero no el de otros.
- El cliente ya tiene ~40.000 ítems y ~1.200 empleados activos, y anticipa reportes mensuales de "ítems más prestados" y "empleados con préstamos vencidos".

Entregable

Responder en este mismo archivo (sección "Respuesta A" abajo), sin escribir código todavía:

1. **Modelo de datos:** lista de modelos propuestos, con sus campos principales (nombre, tipo, relación) y por qué cada uno es un modelo nuevo y no un campo de un modelo existente.
2. **Seguridad:** grupos de seguridad propuestos, y para cada modelo qué reglas de registro (`ir.rule`) se necesitan para cumplir "un empleado ve solo su propio historial".
3. **Rendimiento a escala:** dado el volumen (40k ítems, 1.200 empleados), qué decisiones de modelado o indexación tomarías para que la búsqueda de "ítems disponibles por categoría" y el reporte de "empleados con préstamos vencidos" no degraden con el tiempo.
4. **Automatización:** cómo implementarías la detección de vencidos y el envío de notificación (mecanismo concreto de Odoo, no solo "un cron").
5. **Riesgos / supuestos:** qué asumiste que el enunciado no aclara, y cómo eso cambiaría tu diseño si la respuesta real fuera la contraria.

No hay una única respuesta correcta. Se evalúa que la justificación sea coherente con la decisión tomada, no que coincida con una solución de referencia.

2. Parte B — Seguridad (revisión de código)

Enunciado

El siguiente controlador pertenece a un módulo ya en producción. Expone un endpoint para que un partner externo consulte y actualice el estado de sus propios pedidos vía JSON. **Contiene vulnerabilidades deliberadas.** Identificarlas todas y reescribir el archivo corregido.

```
from odoo import http
from odoo.http import request

class PartnerOrderController(http.Controller):

    @http.route('/api/order/status', type='json', auth='public', csrf=False)
    def get_order_status(self, order_id, partner_id):
        query = "SELECT id, state, amount_total FROM sale_order WHERE id = %s AND partner_id = %s" %
        (order_id, partner_id)
        request.env.cr.execute(query)
        row = request.env.cr.fetchone()
        if not row:
            return {'error': 'not found'}
        return {'id': row[0], 'state': row[1], 'amount_total': row[2]}

    @http.route('/api/order/update_state', type='json', auth='public', csrf=False)
    def update_order_state(self, order_id, new_state):
        order = request.env['sale.order'].sudo().browse(int(order_id))
        order.write({'state': new_state})
        return {'result': 'ok'}

    @http.route('/api/order/list', type='json', auth='user')
    def list_orders(self, partner_id):
        orders = request.env['sale.order'].sudo().search([('partner_id', '=', partner_id)])
        return {'orders': [{ 'id': o.id, 'amount_total': o.amount_total } for o in orders]}
```

Preguntas guía (responder por escrito además de corregir el código)

1. ¿Qué pasa si `order_id` o `partner_id` llegan como `"1 OR 1=1"`? ¿Por qué la corrección no es solo `"castear a int"`?
2. `auth='public' + csrf=False` en un endpoint que modifica estado: ¿cuándo es aceptable esta combinación y cuándo no? ¿Qué reemplazo propondrías?
3. `list_orders` usa `auth='user'` pero recibe `partner_id` como parámetro libre. ¿Qué falta para que un usuario autenticado no pueda leer pedidos de otro partner? ¿La solución es una validación en el controlador, una regla de registro, o ambas? Justificar.
4. `update_order_state` acepta cualquier string como `new_state` y hace `sudo()` sin validar transición de estado. ¿Qué invariantes de negocio se rompen si esto llega a producción tal cual?
5. Ninguna de las tres rutas registra quién hizo el cambio. ¿Cómo lo agregarías sin introducir otro problema de seguridad?

Entregable

- El código corregido, como bloque de código dentro de la sección "Respuesta B" abajo (no requiere repositorio ni instalación).
- Respuestas a las 5 preguntas guía, en la misma sección.

3. Parte C — Rendimiento (revisión de código)

Enunciado

El siguiente método pertenece a un cron que corre cada noche sobre `res.partner` (base con ~80.000 partners, ~500.000 `sale.order` asociadas). Tarda 45 minutos y satura la base. Identificar cada problema de rendimiento y reescribirlo.

```
def _cron_update_partner_risk_score(self):
    partners = self.env['res.partner'].search([('customer_rank', '>', 0)])
    for partner in partners:
        orders = self.env['sale.order'].search([('partner_id', '=', partner.id)])
        total_late = 0
        total_amount = 0
        for order in orders:
            invoices = self.env['account.move'].search([
                ('invoice_origin', '=', order.name),
                ('move_type', '=', 'out_invoice'),
            ])
            for inv in invoices:
                total_amount += inv.amount_total
                if inv.invoice_date_due and inv.payment_state != 'paid':
                    total_late += 1

        score = 'high_risk' if total_late > 3 else 'low_risk'
        partner.write({
            'risk_score': score,
            'total_invoiced': total_amount,
        })
    self.env.cr.commit()
```

Preguntas guía

1. Contar cuántas queries SQL dispara este método en el peor caso, en función de N partners y M órdenes promedio por partner. ¿Por qué escala así?
2. ¿Por qué el `self.env.cr.commit()` dentro del loop es un problema, más allá de la regla general de "no comitear manualmente"? ¿Qué pasa si el cron se cae en el partner 40.000 de 80.000?
3. Reescribir usando `_read_group()` / `formatted_read_group()` (la API de agregación vigente en 19 — `read_group()` está deprecado) para evitar el loop anidado de tres niveles. ¿Qué índice(s) de Postgres agregarías (si alguno) para que la query de agregación no haga sequential scan sobre `account_move`?
4. `invoice_origin = order.name` como forma de vincular factura y pedido: ¿qué asume esto que podría no cumplirse (facturas manuales, pedidos renombrados)? ¿Qué campo del ORM estándar de Odoo sería más confiable?

Entregable

- El código corregido, como bloque de código dentro de la sección "Respuesta C" abajo (no requiere repositorio ni instalación).
- Respuestas a las 4 preguntas guía, en la misma sección.

4. Parte D — Preguntas puntuales sobre Odoo 19

Responder en 1-3 líneas cada una, sin necesidad de código. Buscan validar que el candidato conoce los cambios recientes de versión y no solo replica patrones antiguos que aún "funcionan por compatibilidad".

1. Un módulo propio todavía usa `_sql_constraints` heredado de una versión anterior de Odoo. ¿Qué cambia funcionalmente al migrarlo a `models.Constraint()`, más allá de la sintaxis?
2. ¿Por qué `read_group()` quedó deprecado en 18→19 en favor de `_read_group()` / `formatted_read_group()`? ¿Qué diferencia de forma en el resultado devuelto hay que tener en cuenta al migrar una llamada existente?
3. ¿Qué riesgo concreto tiene seguir usando `self._cr` / `self._uid` / `self._context` en vez de `self.env.cr` / `self.env.uid` / `self.env.context` en código nuevo, aunque hoy todavía funcione?
4. En una vista JS/OWL, ¿qué implica que `@odoo-module` ya no sea requerido? ¿Hay algún caso en el que el candidato todavía lo agregaría a propósito?

5. Parte E — OWL (frontend)

Enunciado

El siguiente componente OWL pertenece a un dashboard que lista los ítems de `library.item` con un botón de refresco. **Contiene errores deliberados de uso del framework** (no de sintaxis JS). Identificarlos y reescribir el componente corregido.

```
import { Component, useState } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export class RentalDashboard extends Component {
  static template = "library_rental.RentalDashboard";
  static props = {};

  setup() {
    this.orm = useService("orm");
    this.state = { items: [], loading: true };
    this.loadItems();
  }

  async loadItems() {
    this.state.items = await this.orm.searchRead(
      "library.item", [], ["name", "state"]
    );
    this.state.loading = false;
  }

  onClickRefresh() {
    this.loadItems();
  }
}

<t t-name="library_rental.RentalDashboard">
  <div class="o_rental_dashboard">
    <button t-on-click="onClickRefresh">Refrescar</button>
    <t t-foreach="state.items" t-as="item">
      <div t-esc="item.name"/>
    </t>
  </div>
</t>
```

Preguntas guía (responder por escrito además de corregir el código)

1. `this.state = { items: [], loading: true }` se asigna como objeto plano, no con `useState()`. ¿Qué consecuencia concreta tiene esto en el render? ¿Por qué el botón "Refrescar" parece no hacer nada aunque `loadItems()` sí se ejecute?
2. `this.loadItems()` se llama directamente desde `setup()`. ¿Qué hook de ciclo de vida de OWL es el lugar correcto para una carga de datos inicial contra el ORM, y por qué llamarlo desde `setup()` sin `await/hook` es un problema (piense en qué pasa si el componente se desmonta antes de que la promesa resuelva)?
3. `loadItems()` no maneja el caso en que `this.orm.searchRead()` rechace la promesa (por ejemplo, un `AccessError` del backend). ¿Qué le pasa hoy al usuario en ese caso? ¿Qué servicio de Odoos usarías para mostrar el error de forma consistente con el resto de la UI?
4. El `t-foreach` no tiene `t-key`. ¿Qué problema concreto de reconciliación del virtual DOM puede causar esto si la lista de ítems cambia de orden entre refrescos?
5. ¿Por qué `static props = {}` es una buena práctica aunque el componente no reciba props hoy? ¿Qué pasaría si otro desarrollador empezara a pasarle un prop sin que el candidato lo notara, con y sin esa declaración?

Entregable

- El componente y template corregidos, como bloques de código dentro de la sección "Respuesta E" abajo (no requiere entorno ni build).
- Respuestas a las 5 preguntas guía, en la misma sección.

6. Respuestas del candidato

Completar debajo. No borrar los enunciados de arriba.

Respuesta A

(completar)

Respuesta B

- 1.
- 2.
- 3.
- 4.
- 5.

Código corregido:

```
# pegar aquí
```

Respuesta C

- 1.
- 2.
- 3.
- 4.

Código corregido:

```
# pegar aquí
```

Respuesta D

- 1.
- 2.
- 3.
- 4.

Respuesta E

- 1.
- 2.
- 3.
- 4.
- 5.

Código corregido:

```
// pegar aquí  
<!-- pegar aquí -->
```